Assignment

# C# Classes, Methods, Properties, Inheritance, and Interfaces

**Generative AI (GenAI) Use**

**GenAI Level 1 — Limited AI Use** ⚠️

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

---

**Scenario — Smart Parcel Delivery Management System**

A local courier company is developing a small application to manage packages that are being delivered to customers. Different types of deliveries have different costs and behaviors, but all deliveries share common information such as a tracking number, destination, weight, and delivery status.

Your task is to develop a **C# console application** that models this system using the object-oriented programming concepts covered in class.

The assignment is intended to demonstrate your ability to design classes that combine **state and behavior**, apply **encapsulation**, and use **inheritance, abstract classes, and interfaces** appropriately. These concepts reflect the progression presented in the course material from fields and properties through classes, inheritance, abstract classes, and interfaces.

**Part 1 — Create the Base Delivery Class**

Create a class named **`Delivery`**.

Each delivery must contain appropriate data for:

- Tracking number
- Recipient name
- Destination
- Weight
- Delivery status

Decide which members should be implemented as **private fields** and which should be exposed through **properties**. Do not simply make all fields public. The course material emphasizes encapsulation by keeping implementation details private and exposing data through controlled members such as properties.

At least one property must use a **private backing field with validation**. For example, a package weight should not be allowed to contain an invalid negative value. Other appropriate properties may use auto-implemented properties. This follows the distinction in the material between auto-properties and backing fields used when validation or other behavior is required.

Include appropriate methods to represent delivery behavior, such as displaying delivery information or updating its status.

**Part 2 — Methods and Parameters**

Add methods that demonstrate the different parameter techniques covered in class.

Your application must contain practical examples of:

- regular parameters passed by value;
- a **`ref`** parameter that modifies an existing value;
- an **`out`** parameter that returns an additional result;
- an **`in`** parameter for a value that should be read without assigning a new value to the parameter;
- an optional parameter with a default value;
- named arguments when calling an appropriate method;
- a **`params`** parameter that allows a variable number of arguments.

The use of each technique must make sense within the parcel-delivery scenario rather than being added as unrelated demonstration code. The course material distinguishes **`ref`**, **`out`**, and **`in`** according to whether the method may modify, produce, or read a value.

**Part 3 — Static and Instance Members**

Add an appropriate **static field or property** that keeps track of the total number of delivery objects created.

Create several delivery objects and demonstrate that:

- each object maintains its own instance state; and
- the static member is shared by all instances.

The distinction between per-object instance fields and type-level static fields is specifically covered in the course material.

**Part 4 — Inheritance**

Extend your application by creating at least **two specialized delivery classes**, for example:

`StandardDelivery`
`ExpressDelivery`

Both must inherit from an appropriate common base class.

Add properties or methods that make the two delivery types meaningfully different. For example, an express delivery may have an additional priority fee or different delivery-time calculation.

Demonstrate inherited members as well as behavior specific to each derived class. Remember that C# classes can directly inherit from only one base class.

**Part 5 — Abstract Class and Overriding**

Refactor the design so that the common delivery type is an **abstract class** where appropriate.

Create at least one **abstract member** representing behavior that must be implemented differently by different delivery types. For example:

`CalculateDeliveryCost()`

Each concrete delivery class must provide its own implementation using `override`.

Your program must demonstrate that the abstract base class cannot be instantiated directly and that its concrete derived classes implement the required behavior. The material defines an abstract class as a base design that cannot be instantiated directly and may leave abstract members for derived classes to implement.

**Part 6 — Interface**

Create an interface representing an additional capability of a delivery. Choose a meaningful name and behavior appropriate to your design.

For example, you could model a capability such as tracking, insurance, notification, or another relevant operation.

At least one of your classes must implement the interface and provide the required behavior.

Your implementation should demonstrate the important distinction that an **interface defines a contract**, while a class provides state and implementation.

**Part 7 — Reference-Type Behavior**

Create one delivery object and assign its reference to a second variable.

Modify an appropriate property through the second variable and then display the property through the first variable.

Use the program output to demonstrate that both variables refer to the **same object**, rather than two independent objects. This behavior is described in the course material: assigning one class variable to another copies the reference, so modifications through one reference can be visible through the other.

**Part 8 — Complete Application and Testing**

Your **`Main`** program must create and test several delivery objects. Do not submit classes that are never exercised.

Your application should demonstrate, at minimum:

1. Creation of several delivery objects.
2. Valid and invalid property assignments to demonstrate validation.
3. Different delivery types.
4. Calculation of delivery costs.
5. The required parameter-passing techniques.
6. Static versus instance members.
7. Inheritance and overridden behavior.
8. Interface implementation.
9. Reference-type behavior.
10. Clear console output showing that each required feature works.

Use realistic test data. Your program should compile and execute without errors.

---

**Report and Practical Evidence**

Submit a **report to Moodle** documenting your completed application.

The practical implementation is the main component of this assignment. **Simply describing the concepts is not sufficient.** The requirements above must be implemented and demonstrated in your own C# application.

Your report must include a **comprehensive set of screenshots** documenting the practical work. Screenshots should clearly show your project structure and relevant .cs files, important class/property/method implementations, inheritance and interface implementations, parameter techniques, validation tests, application execution, and console output demonstrating the required functionality.

Screenshots should provide enough evidence for the professor to follow the development and verify that **all practical requirements have been completed**. A small number of screenshots showing only the final output is not sufficient.

Include a short explanation with the screenshots identifying what is being demonstrated.

**Submission Requirements**

Submit the following through **Moodle**:

- Complete Visual Studio C# project/source code.
- Assignment report.
- Comprehensive screenshots of the practical implementation and testing.
- Required **GenAI Disclosure Statement**, whether or not GenAI was used.

**Respect the due date specified on Moodle.**

In addition to the Moodle submission, **you must present and demonstrate your assignment to the professor to receive a grade.** You should be prepared to run the application, explain your class design, demonstrate the requested functionality, and answer questions about your implementation.

**An assignment that is submitted to Moodle but is not presented to the professor will not be graded.**

---

**Rubric**

| Criterion | Weight |
|---|---|
| **Classes, Fields, Properties & Encapsulation** — Appropriate class design, fields/properties, validation, access modifiers, static/instance members | **25%** |
| **Methods & Parameters** — Correct and meaningful use of methods, value parameters, **`ref`**, **`out`**, **`in`**, optional/named arguments, and **`params`** | **20%** |

| | |
|---|---|
| **Inheritance, Abstract Classes & Interfaces** — Appropriate hierarchy, abstract members, overriding, and interface implementation | **25%** |
| **Application Functionality & Testing** — Complete working application, realistic objects/data, reference-type demonstration, meaningful testing and output | **15%** |
| **Report, Screenshots & Presentation** — Clear report, comprehensive practical screenshots, explanations, organization, and successful demonstration to professor | **15%** |
| **Total** | **100%** |

**Important:** All submitted code must be understood by the student. During the presentation, you may be asked to explain or modify any part of your implementation.